Trabajo Práctico 7

Introducción

Inciso 1

¿Qué servicios presta la capa de red? ¿Cuál es la PDU en esta capa? ¿Qué dispositivo es considerado sólo de la capa de red?

Respuesta:

- Su función es transportar paquetes desde un host emisor a un host receptor atravesando una serie de routers intermedios. Sus dos funciones principales son:
  - **Reenvío** (**Forwarding**)**:** Es la **acción local** que realiza un router para trasladar un paquete desde su interfaz de entrada hacia la interfaz de salida adecuada (operación del plano de datos).
  - **Enrutamiento** (**Routing**)**:** Es el **proceso global** de la red encargado de calcular y determinar la ruta o camino completo que seguirán los paquetes desde el origen hasta el destino (operación del plano de control).
- La **PDU** (**Protocol Data Unit**) o unidad de datos de protocolo en esta capa se denomina **datagrama** (o paquete de la capa de red).
- El dispositivo característico de esta capa es el **router** (o conmutador de capa 3).
  - **Diferencia con otros dispositivos:** Mientras que los switches de la capa de enlace operan únicamente hasta la capa 2 (enlace) y los hosts ejecutan la pila completa de 5 capas, los routers procesan los paquetes hasta la **capa de red** (**capa 3**), examinando las direcciones IP de los datagramas para determinar la interfaz de salida

Inciso 2

¿Por qué se lo considera un protocolo de mejor esfuerzo?

Respuesta:

El protocolo **IP** se considera un servicio de "**mejor esfuerzo**" (**best-effort**) porque realiza todo el intento posible para transportar los datagramas desde

el origen hasta el destino, pero **no ofrece ningún tipo de garantía** sobre la entrega o la calidad del transporte.
Específicamente, carece de las siguientes garantías:

- **Sin garantía de entrega:** Los datagramas pueden perderse o ser descartados en el camino debido a la congestión en las colas de los routers o errores en los enlaces.
- **Sin garantía de orden:** Al enrutarse cada paquete de manera independiente, pueden tomar rutas distintas y llegar desordenados al host receptor[1][2].
- **Sin garantía de integridad:** Los bits del contenido pueden alterarse o corromperse durante la transmisión sin que la capa de red se encargue de corregirlos.
- **Sin garantías de retardo ni ancho de banda:** No asegura un tiempo límite de tránsito extremo a extremo ni reserva un ancho de banda mínimo garantizado para el tráfico.

Inciso 3

¿Cuántas redes clase A, B y C hay? ¿Cuántos hosts como máximo pueden tener cada una?

Respuesta:

Clase A

- **Estructura del primer octeto:** Comienza con el bit `0` (`0xxxxxxx`), dejando 7 bits para identificar redes y 24 bits para identificadores de host.
- **Cantidad de redes:** $2^7 = \mathbf{128}$ redes.
  - *Aclaración:* En la práctica, existen **126 redes utilizables**, ya que la red `0.0.0.0/8` (utilizada para la ruta por defecto) y la red `127.0.0.0/8` (reservada para el bucle local o *loopback*) tienen propósitos especiales6.
- **Máximo de hosts por red:** $2^{24} - 2 = \mathbf{16.777.214\text{ hosts}}$. Se restan 2 direcciones porque la primera representa la dirección de red y la última la dirección de broadcast.
- **Clase B**
- **Estructura del primer octeto:** Comienza con los bits `10` (`10xxxxxx`), dejando 14 bits para red y 16 bits para host.

- **Cantidad de redes:** $2^{14} = 16.384$.
- **Máximo de hosts por red:** $2^{16} - 2 = 65.534$ *hosts*.
  Clase C
- **Estructura del primer octeto:** Comienza con los bits `110` (`110xxxxx`), dejando 21 bits para red y 8 bits para host.
- **Cantidad de redes:** $2^{21} = \mathbf{2.097.152}$ redes.
- **Máximo de hosts por red:** $2^{8} - 2 = \mathbf{254}$ hosts

Inciso 4

¿Qué son las subredes? ¿Por qué es importante siempre especificar la máscara de subred asociada?

Respuesta:

Una **subred** (*subnetwork* o *subnet*) es una división lógica de una red IP física o de un bloque de direcciones IP en múltiples segmentos o redes más pequeñas.

El proceso de división (*subnetting*) consiste en tomar bits prestados de la porción de host de una dirección IP para utilizarlos como identificadores de subred.

Es importante especificar la máscara de la subred porque:

Una dirección IP por sí sola es simplemente un número de 32 bits que no indica dónde termina la identificación de red y dónde empieza la de host. La **máscara de subred** (o el prefijo `/x` en notación CIDR) es imprescindible por las siguientes razones:

1. **Delimita la porción de red y la de host:** Define de forma exacta cuáles bits identifican a la red/subred y cuáles a las interfaces individuales de los host.
2. **Determina si el destino es local o remoto:** Cuando un host emisor quiere enviar un datagrama, aplica su máscara para comparar su propia dirección IP con la de destino.
   - Si están en la **misma subred**, la entrega se hace directamente en la LAN utilizando ARP.
   - Si están en **subredes distintas**, deduce que debe enviar la trama a la dirección MAC del router de salida (*default gateway*).
3. **Identifica los límites de la red (Dirección de subred y Broadcast):** Permite calcular de forma unívoca la **dirección de subred** (primer IP del

rango) y la **dirección de broadcast** (última IP del rango), delimitando las direcciones IP utilizables para los hosts.

4. **Permite el enrutamiento sin clases (CIDR / VLSM):** En las redes modernas no existen máscaras fijas ni implícitas por clase. Sin la máscara, los routers no pueden construir sus tablas de enrutamiento ni aplicar reglas de coincidencia por prefijo más largo (*longest prefix match*).

Inciso 5

¿Cuál es la finalidad del campo Protocol en la cabecera IP? ¿A qué campos de la capa de transporte se asemeja en su funcionalidad?

Respuesta:

El campo **Protocol** (un campo de 8 bits en la cabecera IPv4) tiene como finalidad **identificar a qué protocolo de la capa superior debe entregarse la carga útil** (**payload**) **del datagrama** cuando este llega a su host de destino final.
Cumple la función de **desmultiplexar el tráfico en la capa de red**:

- Cuando la capa IP del receptor recibe un datagrama, examina este número para determinar qué módulo de la pila debe procesar el segmento o mensaje encapsulado.
- Valores característicos incluyen:
  - **6 (0x06):** Indica que la carga útil se debe entregar a **TCP**.
  - **17 (0x11):** Indica que se debe entregar a **UDP**.
  - **1 (0x01):** Indica que la carga útil contiene un mensaje de control **ICMP**.
    Se asemeja funcionalmente a los **campos de número de puerto** (específicamente al **número de puerto de destino**) presentes en las cabeceras de la capa de transporte (TCP/UDP).

Analogía de multiplexación/desmultiplexación entre capas:

- **Campo Protocol (Capa de Red):** Sirve de puente entre la **capa de red y la capa de transporte**, permitiendo al host de destino dirigir el datagrama al protocolo correcto (TCP, UDP, ICMP, etc.).
- **Número de Puerto de Destino (Capa de Transporte):** Sirve de puente entre la **capa de transporte y la capa de aplicación**, permitiendo al host

dirigir el segmento al socket/proceso de aplicación adecuado (por ejemplo, puerto 80 para un servidor web HTTP o puerto 53 para DNS).

División en subredes

Inciso 6

Para cada una de las siguientes direcciones IP (172.16.58.223/26, 163.10.5.49/27, 128.10.1.0/23, 10.1.0.0/24, 8.40.11.179/12) determine:

1. ¿De qué clase de red es la dirección dada (Clase A, B o C)?
2. ¿Cuál es la dirección de subred?
3. ¿Cuál es la cantidad máxima de hosts que pueden estar en esa subred?
4. ¿Cuál es la dirección de broadcast de esa subred?
5. ¿Cuál es el rango de direcciones IP válidas dentro de la subred?

Determinar clase de red

Obtener subred

Inciso 6.1

- 172.16.58.223/26 es red de clase B
- 163.10.5.49/27 es red de clase B
- 128.10.1.0/23 es red de clase B
- 10.1.0.0/24 es red de clase A
- 8.40.11.179/12 es red de clase A

IP: 172.16.58.223 Máscara 26

Inciso 6.2

Conversión octeto por octeto:

- **Dirección IP (172.16.58.223):**
  - `172` = `10101100`
  - `16` = `00010000`
  - `58` = `00111010`
  - `223` = `11011111`
- **Máscara /26 (255.255.255.192):**
  - `255` = `11111111`

- `255` = `11111111`
- `255` = `11111111`
- `192` = `11000000`

Desarrollo de la operación AND:

```
IP (binario):      10101100 . 00010000 . 00111010 . 11011111
Máscara (/26):     11111111 . 11111111 . 11111111 . 11000000
-----------------------------------------------------------
AND (Subred):      10101100 . 00010000 . 00111010 . 11000000
```

Inciso 6.3

Cantidad de hosts es 62. 2^6 -2 = 62

Inciso 6.4

La **dirección de broadcast** representa la última dirección del bloque y se utiliza para enviar mensajes a todos los hosts de esa subred.

Regla teórica en binario:

Se obtiene tomando la dirección de subred y **fijando en 1 todos los bits correspondientes a la porción de host**. En un prefijo `/26`, los primeros 26 bits identifican la red/subred y los últimos **6 bits** pertenecen al host.

Cálculo en binario (Operación OR con la máscara invertida / Wildcard):

Fijamos en `1` los últimos 6 bits del último octeto sobre la dirección de subred:

```
Subred (binario):    10101100 . 00010000 . 00111010 . 11000000
Bits de Host en 1:   00000000 . 00000000 . 00000000 . 00111111
-----------------------------------------------------------------
Broadcast (binario): 10101100 . 00010000 . 00111010 . 11111111
```

172.16.58.255

Inciso 6.5

Mínima: 172.16.58.193
Máxima 172.16.58.254

IP: 163.10.5.49 Máscara: 27

Inciso 6.2

¿Cuál es la dirección de subred? 163.10.5.32 (o `163.10.5.32/27`).

Inciso 6.3

¿Cuál es la cantidad máxima de hosts que pueden estar en esa subred? 30 hosts utilizables (calculado como $2^5 - 2 = 30$, dado que quedan 5 bits para la porción de host).

Inciso 6.4

¿Cuál es la dirección de broadcast de esa subred? 163.10.5.63

Inciso 6.5

¿Cuál es el rango de direcciones IP válidas dentro de la subred? Desde **163.10.5.33** hasta **163.10.5.62**.

IP: 128.10.1.0 Máscara: 23

Inciso 6.2

¿Cuál es la dirección de subred? 128.10.0.0 (o `128.10.0.0/23`). *(Al aplicar la máscara /23 o 255.255.254.0__, el salto en el tercer octeto es de a 2, por lo que el bloque abarca desde 128.10.0.0 hasta 128.10.1.255__).*

Inciso 6.3

¿Cuál es la cantidad máxima de hosts que pueden estar en esa subred? 510 hosts utilizables (calculado como $2^9 - 2 = 510$, dado que quedan 9 bits dedicados a la porción de host).

Inciso 6.4

¿Cuál es la dirección de broadcast de esa subred? 128.10.1.255

Inciso 6.5

¿Cuál es el rango de direcciones IP válidas dentro de la subred? Desde **128.10.0.1** hasta **128.10.1.254**.

IP: 10.1.0.0 Máscara: 24

Inciso 6.2

¿Cuál es la dirección de subred? 10.1.0.0 (o `10.1.0.0/24`).

Inciso 6.3

¿Cuál es la cantidad máxima de hosts que pueden estar en esa subred? 254 hosts utilizables (calculado como $2^8 - 2 = 254$, dado que quedan 8 bits dedicados a la porción de host).

Inciso 6.4

¿Cuál es la dirección de broadcast de esa subred? 10.1.0.255

Inciso 6.5

¿Cuál es el rango de direcciones IP válidas dentro de la subred? Desde **10.1.0.1** hasta **10.1.0.254**.

IP: 8.40.11.179 Máscara 12

Inciso 6.2

¿Cuál es la dirección de subred? 8.32.0.0 (o `8.32.0.0/12`). *(La máscara /12 es 255.240.0.0__. En el 2.º octeto el salto es de a 16; como la IP tiene 40__, cae en el bloque que inicia en 32 y finaliza en 47__).*

Inciso 6.3

¿Cuál es la cantidad máxima de hosts que pueden estar en esa subred? 1.048.574 hosts utilizables (calculado como $2^{20} - 2 = 1.048.574$, dado que quedan 20 bits dedicados a la porción de host).

Inciso 6.4

¿Cuál es la dirección de broadcast de esa subred? 8.47.255.255

Inciso 6.5

¿Cuál es el rango de direcciones IP válidas dentro de la subred? Desde **8.32.0.1** hasta **8.47.255.254**.

Inciso 7

Su organización cuenta con la dirección 128.50.10.0. Indique:

1. ¿Es una dirección de red o de host?
2. Clase a la que pertenece y máscara de clase.
3. Cantidad de hosts posibles.
4. Se necesitan crear, al menos, 513 subredes. Indique:
    1. Máscara necesaria.
    2. Cantidad de redes asignables.
    3. Cantidad de hosts por subred.
    4. Dirección de la subred 710.
    5. Dirección de broadcast de la subred 710.

Inciso 7.1

1. **Clase de red:**
    - El primer octeto es **128**, el cual pertenece al rango de **Clase B** (`128` a `191`).
2. **Máscara por defecto / natural:**
    - Una red de Clase B tiene una máscara por defecto de **255.255.0.0** (**/16**).
3. **División en porción de Red y de Host:**
    - **Porción de red (16 bits):** Los primeros dos octetos → **128.50**
    - **Porción de host (16 bits):** Los últimos dos octetos → **10.0**
4. **Análisis de los bits de host:**
    - Para que fuera una dirección de red, **todos los 16 bits de la porción de host deberían estar en cero** (**0.0**), dando como dirección de red la `128.50.0.0` .
    - Como la porción de host contiene **10.0** (en binario: `00001010.00000000`), tiene bits en `1` , por lo que identifica a una **interfaz/host específico** dentro de la red `128.50.0.0/16` .

Inciso 7.2

- **Máscara por defecto / natural:**
    - Una red de Clase B tiene una máscara por defecto de **255.255.0.0** (**/16**).

Inciso 7.3

2^16 - 2 = 65.534

Inciso 7.4

- Para obtener al menos 513 subredes, calculamos la potencia de 2 necesaria:
  - $2^9 = 512$ *(no alcanza por 1 subred)*
  - $2^{10} = 1024$ *(alcanza correctamente)*
- Se deben tomar **10 bits prestados** de la porción de host.
- Nueva máscara en CIDR: $16 + 10 =$ **/26**
- **Máscara en decimal punteada: 255.255.255.192**

Cantidad de redes asignables = 2^{10}

Cantidad de host por subred: 6 bits que sobran. 2^6 - 2 = 62

Inciso 7.5

- **Dirección de la subred 710:**
  - **Cálculo en binario:**
    - Para la 710.ª subred tomamos $710 - 1 =$ **709** .
    - Representamos 709 en los 10 bits de subred: **1011000101_2**.
    - Ubicamos estos 10 bits en la estructura de la IP (`128.50.ssssssss.sshhhhhh`) :
      - **3.er octeto (primeros 8 bits):** `10110001` = **177**
      - **4.º octeto (2 bits restantes + 6 ceros de host):** `01000000` = **64**
  - **Dirección de la Subred 710: 128.50.177.64** (o `128.50.177.64/26`).

Inciso 7.6

- **Dirección de broadcast de la subred 710:**
  - Fijamos en `1` los 6 bits de host sobre el 4.º octeto de la subred (`01111111`):
    - **4.º octeto:** `01111111` = **127**
  - **Dirección de Broadcast: 128.50.177.127**

Inciso 8

Si usted estuviese a cargo de la administración del bloque IP 195.200.45.0/24

1. ¿Qué máscara utilizaría si necesita definir al menos 9 subredes?

2. Indique la dirección de subred de las primeras 9 subredes.
3. Seleccione una e indique dirección de broadcast y rango de direcciones asignables en esa subred

Inciso 8.1

La máscara a utilizar es 2^4 = 16

Entonces debemos tomar 4 bits del último octeto

La máscara sería /28

- **Máscara en decimal punteado: 255.255.255.240**
- **Salto de bloque en el 4.º octeto:** $256 - 240 = \mathbf{16}$

Inciso 8.2

Avanzando de 16 en 16 en el 4.º octeto (comenzando desde la subred 0):

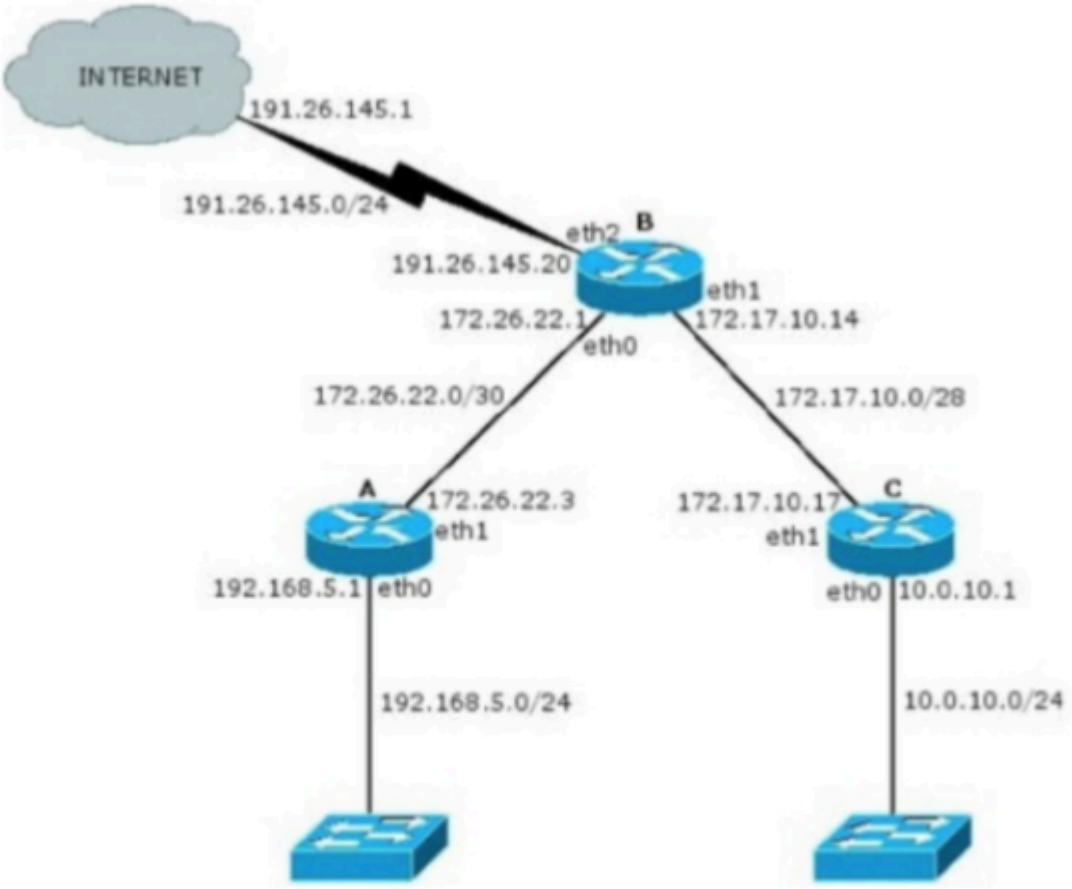
1. **Subred 1: 195.200.45.0/28**
2. **Subred 2: 195.200.45.16/28**
3. **Subred 3: 195.200.45.32/28**
4. **Subred 4: 195.200.45.48/28**
5. **Subred 5: 195.200.45.64/28**
6. **Subred 6: 195.200.45.80/28**
7. **Subred 7: 195.200.45.96/28**
8. **Subred 8: 195.200.45.112/28**
9. **Subred 9: 195.200.45.128/28**

Iciso 8.3

Seleccionamos la **Subred 3** (**195.200.45.32/28**):

- **Dirección de Subred:** `195.200.45.32`
- **Dirección de Broadcast: 195.200.45.47** *(un número antes de que comience la subred siguiente, .48__)*
- **Rango de direcciones IP asignables (hosts válidos):** Desde **195.200.45.33** hasta **195.200.45.46** *(brinda un total de* $2^4 - 2 = \mathbf{14}$ hosts utilizables *por subred)*

Inciso 9



1. Verifique si es correcta la asignación de direcciones IP y, en caso de no serlo, modifique la misma para que lo sea.
2. ¿Cuántos bits se tomaron para hacer subredes en la red 10.0.10.0/24? ¿Cuántas subredes se podrían generar?
3. Para cada una de las redes utilizadas indique si son públicas o privadas.

Inciso 9.1

1. LAN inferior izquierda (Router A eth0 hacia el switch)
   - **Subred declarada:** `192.168.5.0/24`
   - **Rango útil de hosts:** `192.168.5.1` a `192.168.5.254`
   - **IP configurada en Router A (eth0):** `192.168.5.1`
   - **Estado: Correcto.**
2. LAN inferior derecha (Router C eth0 hacia el switch)
   - **Subred declarada:** `10.0.10.0/24`
   - **Rango útil de hosts:** `10.0.10.1` a `10.0.10.254`
   - **IP configurada en Router C (eth0):** `10.0.10.1`
   - **Estado: Correcto.**
3. Enlace WAN punto a punto Router A – Router B

- **Subred declarada:** `172.26.22.0/30`
- **Cálculo de la máscara /30 (bloque de 4 IPs):**
  - Dirección de red: `172.26.22.0`
  - Primer host válido: `172.26.22.1`
  - Segundo host válido: `172.26.22.2`
  - Dirección de broadcast: `172.26.22.3`
- **IPs configuradas en el diagrama:**
  - Router B (eth0): `172.26.22.1`
  - Router A (eth1): `172.26.22.3`
- **Estado: INCORRECTO.**
  - **Error:** Se asignó `172.26.22.3` a la interfaz `eth1` del Router A, pero esa dirección es la dirección de broadcast de la subred `/30` y no puede asignarse a un dispositivo.
  - **Corrección:** Cambiar la IP de **Router A** (**eth1**) a `172.26.22.2`.}

4. Enlace WAN punto a punto Router B – Router C
   - **Subred declarada:** `172.17.10.0/28`
   - **Cálculo de la máscara /28 (bloque de 16 IPs):**
     - Dirección de red: `172.17.10.0`
     - Rango útil de hosts: `172.17.10.1` a `172.17.10.14`
     - Dirección de broadcast: `172.17.10.15`
     - *(La siguiente subred empieza en `172.17.10.16/28`, cuyo rango va de .16 a .31)*
   - **IPs configuradas en el diagrama:**
     - Router B (eth1): `172.17.10.14`
     - Router C (eth1): `172.17.10.17`
   - **Estado: INCORRECTO.**
     - **Error:** La IP `172.17.10.17` pertenece a otra subred (`172.17.10.16/28`), por lo que Router B y Router C no pueden comunicarse directamente en capa 3.
     - **Corrección:** Asignar a **Router C** (**eth1**) cualquier dirección válida dentro del rango `172.17.10.1` – `172.17.10.14` (por ejemplo, `172.17.10.13` o `172.17.10.2`).
5. Enlace hacia Internet (Router B eth2)
   - **Subred declarada:** `191.26.145.0/24`
   - **Rango útil de hosts:** `191.26.145.1` a `191.26.145.254`

- **IPs configuradas:**
  - Extremo Internet: `191.26.145.1`
  - Router B (eth2): `191.26.145.20`
- **Estado: Correcto.**

Inciso 9.2

APUNTE TEÓRICO SUBNETTING -> apunte en base al subido por la cátedra
¿Cuántos bits se tomaron para hacer subredes?

- **Clase por defecto:** La dirección `10.0.10.0` pertenece al rango `0.0.0.0` a `127.255.255.255`, por lo que es originalmente una **red de Clase A**.
- **Máscara original (por defecto de Clase A):** `/8` (`255.0.0.0`), la cual utiliza 8 bits para red y 24 bits para hosts.
- **Máscara actual configurada:** `/24` (`255.255.255.0`), que cuenta con 24 bits de prefijo.
- **Bits tomados para subred** (b_s)**:**

$$\text{Bits de subred} = 24 - 8 = \mathbf{16}\ \text{bits}$$

¿Cuántas subredes se podrían generar?

Aplicando la fórmula de cantidad de subredes a partir de los bits asignados (2^{b_s}):

$$\text{Cantidad de subredes} = 2^{16} = \mathbf{65.536}\ \text{subredes}$$

Por lo tanto:

- Se tomaron **16 bits**.
- Se pueden generar **65.536 subredes** posibles (todas con máscara `/24`, albergando cada una hasta $2^8 - 2 = 254$ hosts útiles).

Inciso 9.3

> **Nota**
>
> Para saber si una red es **pública** o **privada**, debemos fijarnos en el estándar **RFC 1918**. Este documento reserva exclusivamente tres rangos de direcciones IPv4 para uso privado en redes internas o domésticas (no enrutables directamente en la Internet global):

- **Clase A:** `10.0.0.0` a `10.255.255.255` (prefijo `10.0.0.0/8`)
- **Clase B:** `172.16.0.0` a `172.31.255.255` (prefijo `172.16.0.0/12`)
- **Clase C:** `192.168.0.0` a `192.168.255.255` (prefijo `192.168.0.0/16`)
  Cualquier dirección IP que **no** caiga dentro de estos rangos (o de otros rangos reservados especiales como loopback `127.0.0.0/8` o multicast `224.0.0.0/4`) es una **dirección pública**, válida y enrutable en Internet.

La información de la nota se encuentra en la teoría de la cátedra

- No tienen significado global no son únicas.
- Definidas en RFC-1918.
- Se utilizan en Intranets. Redes autónomas sin conexión a Internet.
- Para conectar a Internet requieren un proceso de transformación: NAT, RFC-1631.
- No deberían pasar a la Internet. Filtradas por routers de borde.
  - 10.0.0.0 – 10.255.255.255, 1 Clase A.
  - 172.16.0.0 – 172.31.255.255, 16 Clases B.
  - 192.168.0.0 – 192.168.255.255, 256 Clases C.

Clasificación de las redes del diagrama

1. **Red** `192.168.5.0/24` **(LAN izquierda)**
   - **Tipo: Privada**.
   - **Motivo:** Pertenece al rango privado de Clase C (`192.168.0.0` a `192.168.255.255`).
2. **Red** `10.0.10.0/24` **(LAN derecha)**
   - **Tipo: Privada**.
   - **Motivo:** Pertenece al rango privado de Clase A (`10.0.0.0` a `10.255.255.255`).
3. **Red** `172.26.22.0/30` **(Enlace Router A – Router B)**
   - **Tipo: Privada**.

- **Motivo:** Su segundo octeto es `26`, por lo que cae dentro del rango privado de Clase B (`172.16.0.0` a `172.31.255.255`).

4. **Red `172.17.10.0/28` (Enlace Router B – Router C)**
   - **Tipo: Privada**.
   - **Motivo:** Su segundo octeto es `17`, encontrándose también dentro del rango privado de Clase B (`172.16.0.0` a `172.31.255.255`).
5. **Red `191.26.145.0/24` (Enlace Router B – Internet)**
   - **Tipo: Pública**.
   - **Motivo:** No pertenece a ninguno de los rangos de la RFC 1918; es una red de Clase B pública enrutable hacia y desde Internet.

CIDR

Inciso 10

¿Qué es CIDR (Class Interdomain routing)? ¿Por qué resulta útil?

Respuesta

CIDR (*Classless Inter-Domain Routing* o Enrutamiento Entre Dominios sin Clase) es un esquema de asignación de direcciones e ingeniería de ruteo IP creado para **reemplazar el antiguo sistema de direccionamiento rígido basado en clases** (**Clase A, B y C**).

En lugar de limitar la porción de red a longitudes fijas de 8, 16 o 24 bits, CIDR **permite utilizar máscaras de subred de longitud variable**. Las direcciones se expresan mediante la notación de prefijo **a.b.c.d/x**, donde **x** representa la cantidad exacta de bits dedicados a la porción de red (el prefijo de red) y los `32 - x` bits restantes identifican a los hosts.

¿Por qué resulta útil? (Ventajas principales)

1. **Evita el desperdicio y frena el agotamiento de direcciones IPv4:**
   - Con el esquema con clases, si una organización necesitaba direccionar 1.000 hosts, la Clase C (254 hosts) le quedaba corta y se le debía asignar una Clase B (65.534 hosts), desperdiciando más del 98% de las direcciones.

- Con CIDR se le asigna un bloque de direcciones contiguas a la medida exacta de sus necesidades (por ejemplo, un bloque `/22` que otorga 1.022 IPs útiles).

2. **Permite la agregación / resumen de rutas (Route Summarization):**
   - Permite tomar múltiples subredes o bloques contiguos y **agruparlos para anunciarlos al exterior bajo una sola dirección de "superred" o prefijo común**.
3. **Reduce el tamaño de las tablas de enrutamiento:**
   - Al reducir la cantidad de rutas publicadas en el núcleo de Internet (*core*), las tablas de enrutamiento de los routers no colapsan.
   - Esto trae beneficios directos: los routers requieren **menos memoria**, realizan búsquedas con **menor procesamiento** y se reducen los **retardos de enrutamiento**.
4. **Soporta VLSM (Variable-Length Subnet Masking):**
   - Permite realizar divisiones jerárquicas sucesivas en subredes con máscaras adaptadas a cada necesidad (como usar `/30` para enlaces punto a punto entre routers, evitando malgastar direcciones).

Inciso 11

¿Cómo publicaría un router las siguientes redes si se aplica CIDR?

1. 198.10.1.0/24
2. 198.10.0.0/24
3. 198.10.3.0/24
4. 198.10.2.0/24

Respuesta

1. Ordenar y escribir las direcciones en binario

Las 4 redes son consecutivas: `198.10.0.0`, `198.10.1.0`, `198.10.2.0`, `198.10.3.0`.

Como los primeros dos octetos (`198.10.`) y el último (`.0`) son idénticos, nos enfocamos en el 3.er octeto en binario:

Red	1.er octeto	2.º octeto	3.er octeto (binario)	4.º octeto
`198.10.0.0`	`11000110`	`00001010`	`00000000`	`00000000`

Red	1.er octeto	2.º octeto	3.er octeto (binario)	4.º octeto
`198.10.1.0`	`11000110`	`00001010`	`00000001`	`00000000`
`198.10.2.0`	`11000110`	`00001010`	`00000010`	`00000000`
`198.10.3.0`	`11000110`	`00001010`	`00000011`	`00000000`

2. Armar la máscara con unos donde todos los bits coinciden

- 1.er octeto: todos tienen `11000110` → Coinciden los **8 bits** (`11111111`).
- 2.º octeto: todos tienen `00001010` → Coinciden los **8 bits** (`11111111`).
- 3.er octeto: coinciden los primeros **6 bits** (`000000`), mientras que los últimos 2 difieren (`00`, `01`, `10`, `11`) → Ponemos 6 unos y completamos con ceros (`11111100`).
- 4.º octeto: no se evalúa más allá de donde empezaron a diferir → Todo ceros (`00000000`).

La **máscara resultante** tiene $8 + 8 + 6 = \mathbf{22}$ unos (es decir, prefijo /**22**):

$$\text{Máscara} = 11111111.11111111.11111100.00000000 \quad (255.255.252.0)$$

Operación lógica AND (Cualquier IP AND Máscara)

Se toma cualquiera de las 4 direcciones (por ejemplo, la primera: `198.10.1.0`) y se aplica la operación lógica `AND` bit a bit con la máscara obtenida

```
IP (198.10.1.0):      11000110 . 00001010 . 00000001 . 00000000
Máscara (/22):        11111111 . 11111111 . 11111100 . 00000000
-------------------------------------------------------------- (AND bit a bit)
Resultado:            11000110 . 00001010 . 00000000 . 00000000
```

En decimales es: 198.10.0.0

Resultado

El router publicará la red resultante junto con la máscara obtenida:

198.10.0.0

Inciso 12

Listar las redes involucradas en los siguientes bloques CIDR:

- 200.56.168.0/21
- 195.24.0.0/13
- 195.24/13

Respuesta
Como no me dan a qué máscara destino llegar deduzco que es la clase natural de la dirección IP.
Tanto `200.56.168.0` como `195.24.0.0` tienen su primer octeto en el rango de **192 a 223**, lo que significa que son originalmente direcciones de **Clase C**.

Primer caso (200.56.168.0/21)

Restamos la máscara destino menos la máscara del bloque:

$$\text{Diferencia de bits} = 24 - 21 = 3 \text{ bits}$$

Eso significa que hay $2^3 = \mathbf{8}$ redes metidas adentro

Opcion A para resolverlo

- El prefijo `/21` está en el 3.er octeto (porque los primeros dos octetos son $8 + 8 = 16$ bits).
- Los bits que faltan en ese octeto hasta llegar a 24 son 3 bits ($24 - 21 = 3$).
- Con 3 bits, el número máximo que puedes formar es $2^3 - 1 = 7$ (o sea, del `0` al `7`).
- Se lo suma al octeto base:
  - Empieza en: `168`
  - Termina en: $168 + 7 = \mathbf{175}$
- **Resultado:** Las redes van desde `200.56.168.0/24` hasta `200.56.175.0/24`.

Opción B: En binario

1. Escribimos el tercer octeto original (`168`):

$$168 = \mathbf{10101}000$$

2. Como la máscara es `/21` ($16 + 5$), los primeros **5 bits están congelados** (`10101`).

3. Los **últimos 3 bits** son los que varían para dar todas las redes:
   - La primera red tiene esos 3 bits en **ceros**: `10101 000` → 168 → `200.56.168.0/24`
   - La última red tiene esos 3 bits en **unos**: `10101 111` → 175 → `200.56.175.0/24

Segundo caso (195.24.0.0/13)

1. Determinación de la máscara destino:
   - Al comenzar con 195 (rango 192–223), es una dirección de Clase C.
   - Su máscara por defecto es /24.
2. Cantidad de redes involucradas:
   - Diferencia de bits: 24 - 13 = 11 bits.
   - Cantidad de redes: 2^11 = 2048 redes /24.
3. Análisis binario de los 11 bits variables:
   - El prefijo /13 toma los 8 bits del 1.° octeto y 5 bits del 2.° octeto. -
   - 2.° octeto original (24): 00011000. Los primeros 5 bits (00011) quedan fijos.
   - Los 11 bits que varían corresponden a:
     - Los últimos 3 bits del 2.° octeto.
     - Los 8 bits completos del 3.er octeto.
4. Cálculo de los extremos del rango:
   - Primera red (11 bits en 0): 11000011 . 00011000 . 00000000 . 00000000 = 195.24.0.0/24
   - Última red (11 bits en 1): 11000011 . 00011111 . 11111111 . 00000000 = 195.31.255.0/24 Rango disponible: 195.24.0.0/24 ... 195.31.255.0/24

Tercer caso (195.24/13)

Es exactaente igual al caso anterior.

Inciso 13

El bloque CIDR 128.0.0.0/2 o 128/2, ¿Equivale a listar todas las direcciones de red de clase B? ¿Cuál sería el bloque CIDR que agrupa todas las redes de clase A?

Respuesta

128.0.0.0/2 es de clase B
Tenemos 16/ de máscara para los de clase B

Entonces tenemos

$$\text{Diferencia de bits} = 16 - 2 = 14 \text{ bits}$$

Significa que tenemos 2^{14} redes metidas dentro, que coincide con la cantidad máxima para los de clase B, entonces si, equivale a listar todas las direcciones de clase B.

Justificación teórica a nivel de bits:

- Toda dirección de **Clase B** se define por tener en su primer octeto el patrón binario inicial `10xxxxxx` (un prefijo de 2 bits fijos `10`).
- La dirección `128.0.0.0` en binario comienza exactamente con `10000000`.
- Al aplicar la máscara /**2**, se están fijando exactamente los primeros 2 bits (`10`).
- Como la máscara por defecto de la Clase B es /**16** (16 bits de red), los bits variables de red son $16 - 2 = 14$ bits, lo que genera exactamente $2^{14} = 16.384$ redes de Clase B (desde `128.0.0.0/16` hasta `191.255.0.0/16`).
- Por ende, **sí, equivale exactamente a listar todas las redes de Clase B**.

¿Cuál sería el bloque CIDR que agrupa todas las redes de clase?
El bloque CIDR es: 0.0.0.0/1 (o simplemente 0/1).

- Justificación: Las redes de Clase A tienen su primer bit fijo en '0' (patrón 0xxxxxxx, rango 0.0.0.0 a 127.255.255.255). Una máscara /1 fija ese único bit en 0. Como la máscara por defecto de Clase A es /8, la diferencia es 8 - 1 = 7 bits, agrupando las 2^7 = 128 redes de Clase A posibles.

VLSM

Inciso 14

¿Qué es y para qué se usa VLSM?

Respuesta

VLSM (del inglés *Variable-Length Subnet Masking* o **Máscara de Subred de Longitud Variable**) es una técnica de subneteo que permite aplicar diferentes máscaras de subred a distintos segmentos dentro de un mismo bloque de direcciones IP.

A diferencia del subneteo tradicional (de longitud fija), donde una sola máscara se aplica por igual a todas las subredes, VLSM permite realizar **divisiones sucesivas** (**o sub-subneteo**) para adaptar el tamaño de cada subred a la cantidad de hosts que realmente necesita cada segmento.

¿Para qué se usa VLSM?

1. **Evitar el desperdicio masivo de direcciones IPv4:** Es una de las principales técnicas creadas para mitigar el agotamiento de direcciones IP. Permite ajustar las máscaras para no dejar bloques de direcciones sin utilizar.
2. **Adaptarse a necesidades heterogéneas:** Permite alimentar en una misma red segmentos de tamaños muy distintos. Por ejemplo, dentro de una misma dirección base podés asignar:
   - Una máscara `/25` (126 hosts) para una LAN grande.
   - Una máscara `/28` (14 hosts) para un departamento pequeño.
   - Una máscara `/30` (2 hosts útiles) para un enlace punto a punto entre routers, evitando desperdiciar decenas de direcciones en un cable que solo conecta dos interfaces.
3. **Facilitar el resumen de rutas** (**Route Summarization**)**:** Al permitir un diseño jerárquico de subredes contiguas, VLSM facilita consolidar múltiples subredes en un único prefijo de anunciación, ayudando a mantener compactas las tablas de enrutamiento de los routers.

Inciso 15

Describa, con sus palabras, el mecanismo para dividir subredes utilizando VLSM

EJEMPLO VLSM

Respuesta

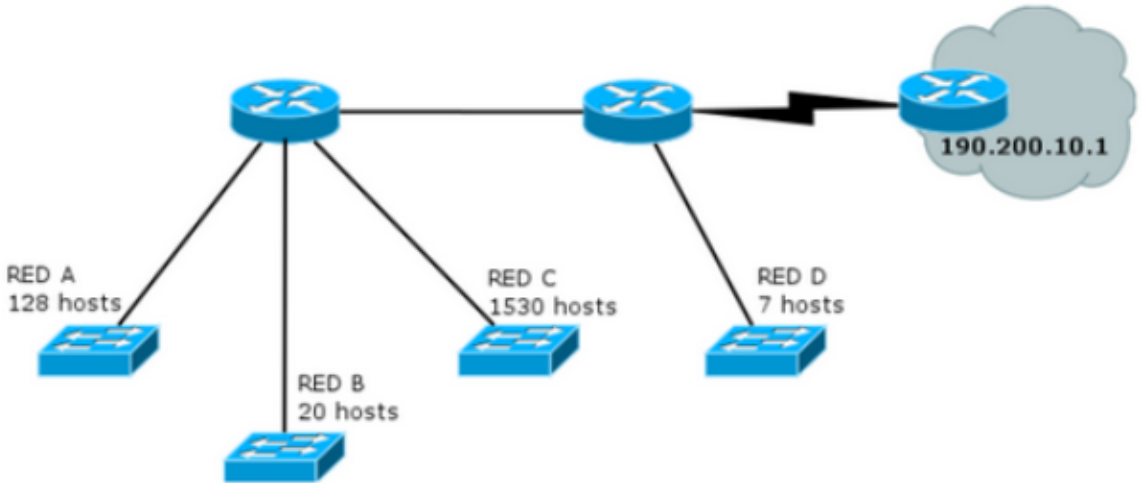
1. Relevar y ordenar los requerimientos de mayor a menor

- Se listan todos los segmentos de red necesarios (redes LAN y enlaces punto a punto) ordenados estrictamente desde el que requiere **mayor cantidad de hosts** hasta el más pequeño.

2. Calcular los bits de host para el requerimiento más grande
   - Se calcula la cantidad de bits de host (h) necesarios para cubrir el requerimiento actual ($2^h - 2 \geq$ hosts) y se determina la máscara de subred correspondiente para este bloque.
3. Realizar la primera división y asigna
   - Se asigna la primera de las subredes obtenidas al segmento en análisis, mientras que la subred contigua que queda libre se reserva para volver a dividirla.
4. Dividir de forma recursiva (sub-subnetear)
   - Se toma la subred libre que quedó del paso previo y se vuelve a dividir aplicando una máscara más larga (más específica), ajustada a las necesidades del siguiente requerimiento en la lista.
   - Se asigna una de las nuevas subredes creadas y se toma la otra libre para continuar el proceso.
5. Repetir hasta completar todos los enlaces
   - El ciclo se itera sucesivamente hasta asignar todas las LANs y los enlaces punto a punto entre routers (los cuales suelen finalizar con máscara /**30** o `255.255.255.252` por requerir solo 2 hosts útiles)

Inciso 16

Suponga que trabaja en una organización que tiene la red que se ve en el gráfico y debe armar el direccionamiento para la misma, minimizando el desperdicio de direcciones IP.
Dicha organización posee la red 205.10.192.0/19, que es la que usted deberá utilizar.



1. ¿Es posible asignar las subredes correspondientes a la topología utilizando subnetting sin VLSM? Indique la cantidad de hosts que se desperdicia en cada subred.
2. Asigne direcciones a todas las redes de la topología. Tome siempre en cada paso la primera dirección de red posible.
3. Para mantener el orden y el inventario de direcciones disponibles, haga un listado de todas las direcciones libres que le quedaron, agrupándolas utilizando CIDR.
4. Asigne direcciones IP a todas las interfaces de la topología que sea posible.

Inciso 16.1

Sin usar VLSM, todas las subredes deben tener la misma máscara. Por la regla de la guía: la máscara la define la subred que más hosts necesita (en este caso, la **Red C con 1530 hosts**).

$2^{b_h} - 2 \geq 1531 \implies b_h = 11$ bits para hosts ($2^{11} - 2 = 2048 - 2 =$ **2046** hosts útiles

La máscara fija para TODAS las subredes será = 32 - 11 = /21

¿Alcanza el bloque original `205.10.192.0/19`?

- El bloque original tiene máscara /**19**.
- Al dividirlo en subredes /**21**, los bits para subred son:

$$b_s = 21 - 19 = 2 \text{ bits}$$

- Cantidad de subredes que se pueden generar con máscara fija:

$$\text{Cantidad de subredes} = 2^2 = \mathbf{4} \text{ subredes}$$

En el hipotético caso de que el ejercicio ignorara el enlace inter-router y solo considerara las 4 LANs (A, B, C y D), **sí sería posible** asignar una subred `/21` a cada una.

En ese escenario, **el desperdicio de hosts en cada subred** (Capacidad útil: 2046 hosts) sería:

- **Red C (1530 hosts + 1 router = 1531 usados):**

$$2046 - 1531 = \textbf{515} \text{ hosts desperdiciados}$$

- **Red A (128 hosts + 1 router = 129 usados):**

$$2046 - 129 = \textbf{1917} \text{ hosts desperdiciados}$$

- **Red B (20 hosts + 1 router = 21 usados):**

$$2046 - 21 = \textbf{2025} \text{ hosts desperdiciados}$$

- **Red D (7 hosts + 1 router = 8 usados):**

$$2046 - 8 = \textbf{2038} \text{ hosts desperdiciados}$$

Inciso 16.2

Paso 1: Subnetear para RED C (1530 hosts)

- Requerimiento: $2^{11} - 2 = 2046 \geq 1530 \implies b_h = 11 \implies$ Máscara /**21** (`255.255.248.0`)
- **MÁSCARA DE RED C** = `11111111 . 11111111 . 11111000 . 00000000` (/21)
- **DIRECCIÓN ASIGNADA A RED C:** `11001101 . 00001010 . 11000000 . 00000000` → `205.10.192.0/21`
- **PRÓXIMO A DIVIDIR:** `11001101 . 00001010 . 11001000 . 00000000` (`205.10.200.0/21`)

Paso 2: Subnetear para RED A (128 hosts)

- Requerimiento: $2^{8} - 2 = 254 \geq 128 \implies b_h = 8 \implies$ Máscara /**24** (`255.255.255.0`)
- **MÁSCARA DE RED A** = `11111111 . 11111111 . 11111111 . 00000000` (/24)

- **DIRECCIÓN ASIGNADA A RED A:** `11001101 . 00001010 . 11001000 . 00000000` → `205.10.200.0/24`
- **PRÓXIMO A DIVIDIR:** `11001101 . 00001010 . 11001001 . 00000000` (`205.10.201.0/24`)

Paso 3: Subnetear para RED B (20 hosts)

- Requerimiento: $2^5 - 2 = 30 \geq 20 \implies b_h = 5 \implies$ Máscara /**27** (`255.255.255.224`)
- **MÁSCARA DE RED B** = `11111111 . 11111111 . 11111111 . 11100000` (/27)
- **DIRECCIÓN ASIGNADA A RED B:** `11001101 . 00001010 . 11001001 . 00000000` → `205.10.201.0/27`
- **PRÓXIMO A DIVIDIR:** `11001101 . 00001010 . 11001001 . 00100000` (`205.10.201.32/27`)

Paso 4: Subnetear para RED D (7 hosts)

- Requerimiento: $2^4 - 2 = 14 \geq 7 \implies b_h = 4 \implies$ Máscara /**28** (`255.255.255.240`)
- **MÁSCARA DE RED D** = `11111111 . 11111111 . 11111111 . 11110000` (/28)
- **DIRECCIÓN ASIGNADA A RED D:** `11001101 . 00001010 . 11001001 . 00100000` → `205.10.201.32/28`
- **PRÓXIMO A DIVIDIR:** `11001101 . 00001010 . 11001001 . 00110000` (`205.10.201.48/28`)

Paso 5: Subnetear para el Enlace Punto a Punto (entre routers internos - 2 hosts)

- Requerimiento: $2^2 - 2 = 2 \geq 2 \implies b_h = 2 \implies$ Máscara /**30** (`255.255.255.252`)
- **MÁSCARA DE RED P-P** = `11111111 . 11111111 . 11111111 . 11111100` (/30)
- **DIRECCIÓN ASIGNADA AL ENLACE P-P:** `11001101 . 00001010 . 11001001 . 00110000` → `205.10.201.48/30`

Inciso 16.3

Regla para CIDR

Detalle por cada etapa en binario de los bloques hermanos que quedaron libres y su posterior agregación CIDR:

- **En el nivel `/19` (al subnetear en `/21` para Red C):**
  - Subredes libres:
    `11001101 . 00001010 . 11010000 . 00000000` (`205.10.208.0/21`)
    `11001101 . 00001010 . 11011000 . 00000000` (`205.10.216.0/21`) `----------`
  - **NÚMERO CIDR** = `11001101 . 00001010 . 11010000 . 00000000` → `205.10.208.0/20`
- **En el nivel `/21` (al subnetear en `/24` para Red A):**
  - Subredes libres (quedan 6 redes `/24`): `11001101 . 00001010 . 11001010 . 00000000` (`205.10.202.0/24`) `11001101 . 00001010 . 11001011 . 00000000` (`205.10.203.0/24`) `11001101 . 00001010 . 11001100 . 00000000` (`205.10.204.0/24`) `11001101 . 00001010 . 11001101 . 00000000` (`205.10.205.0/24`) `11001101 . 00001010 . 11001110 . 00000000` (`205.10.206.0/24`) `11001101 . 00001010 . 11001111 . 00000000` (`205.10.207.0/24`) `----------`
  - **NÚMEROS CIDR** =
    `11001101 . 00001010 . 11001010 . 00000000` → `205.10.202.0/23` *(agrupa 202 y 203)*
    `11001101 . 00001010 . 11001100 . 00000000` → `205.10.204.0/22` *(agrupa 204 a 207)*
- **En el nivel `/24` (al subnetear en `/27` para Red B):**
  - Subredes libres (quedan 6 redes `/27`): `11001101 . 00001010 . 11001001 . 01000000` (`205.10.201.64/27`) `11001101 . 00001010 . 11001001 . 01100000` (`205.10.201.96/27`) `11001101 . 00001010 . 11001001 . 10000000` (`205.10.201.128/27`) `11001101 . 00001010 . 11001001 . 10100000` (`205.10.201.160/27`) `11001101 . 00001010 . 11001001 . 11000000` (`205.10.201.192/27`) `11001101 . 00001010 . 11001001 . 11100000` (`205.10.201.224/27`) `----------`

- **NÚMEROS CIDR** = `11001101 . 00001010 . 11001001 . 01000000` → `205.10.201.64/26` *(agrupa .64 y .96)* `11001101 . 00001010 . 11001001 . 10000000` → `205.10.201.128/25` *(agrupa .128 a .224)*

- **En el nivel `/28` (al subnetear en `/30` para el Enlace P-P):**
  - Subredes libres (quedan 3 redes `/30`): `11001101 . 00001010 . 11001001 . 00110100` (`205.10.201.52/30`) `11001101 . 00001010 . 11001001 . 00111000` (`205.10.201.56/30`) `11001101 . 00001010 . 11001001 . 00111100` (`205.10.201.60/30`) `--------------------------------------------------`
  - **NÚMEROS CIDR** = `11001101 . 00001010 . 11001001 . 00110100` → `205.10.201.52/30`
    `11001101 . 00001010 . 11001001 . 00111000` → `205.10.201.56/29` *(agrupa .56 y .60)*

Listado Final de Bloques Libres (Inventario CIDR):

1. `205.10.201.52/30`
2. `205.10.201.56/29`
3. `205.10.201.64/26`
4. `205.10.201.128/25`
5. `205.10.202.0/23`
6. `205.10.204.0/22`
7. `205.10.208.0/20`

Inciso 16.4

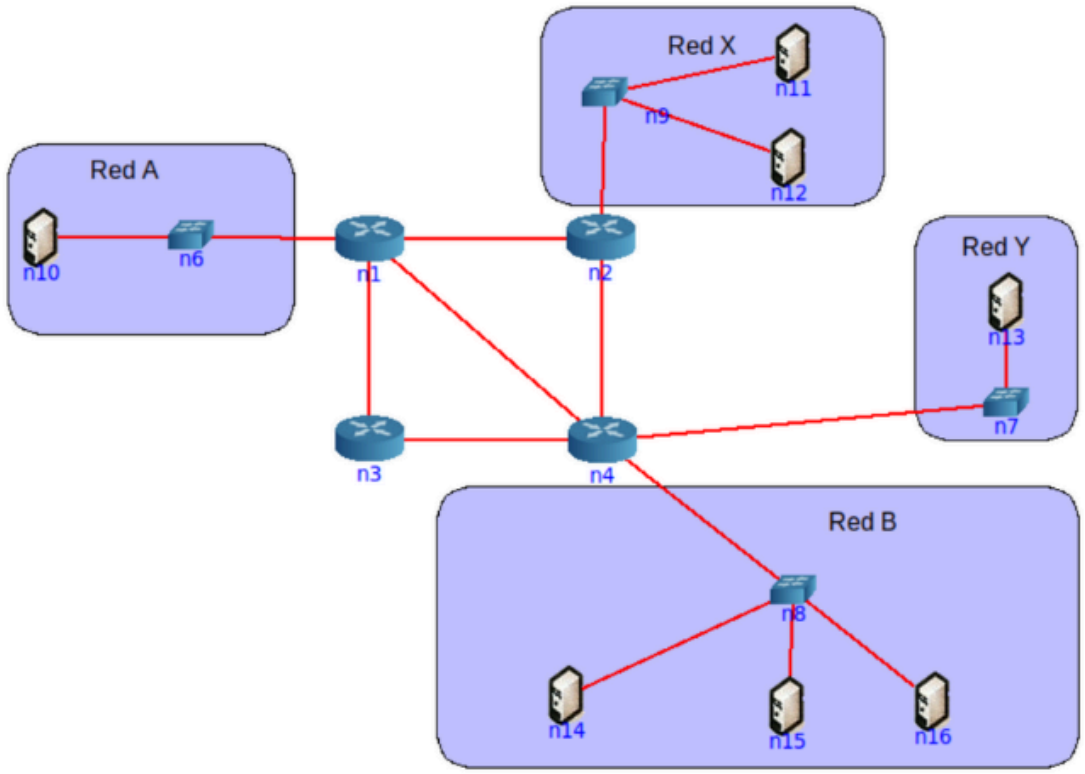
Debemos asignar la **primera IP útil** de cada subred a la interfaz del router en cada LAN, y direcciones contiguas en los enlaces punto a punto:

- **Router 1 (Router izquierdo):**
  - Interfaz LAN RED C: `205.10.192.1 /21`
  - Interfaz LAN RED A: `205.10.200.1 /24`
  - Interfaz LAN RED B: `205.10.201.1 /27`
  - Interfaz hacia Router 2 (PTP): `205.10.201.49 /30`
- **Router 2 (Router central):**
  - Interfaz LAN RED D: `205.10.201.33 /28`

- Interfaz hacia Router 1 (PTP): `205.10.201.50 /30`
- Interfaz hacia Router de la nube (ISP): `190.200.10.2` (siguiente dirección IP del enlace público en el que el router externo tiene configurada la `190.200.10.1`).

Inciso 17

Utilizando la siguiente topología y el bloque asignado, arme el plan de direccionamiento IPv4 teniendo en cuenta las siguientes restricciones:



1. Utilizar el bloque IPv4 200.100.8.0/22.
2. La red A tiene 125 hosts y se espera un crecimiento máximo de 20 hosts.
3. La red X tiene 63 hosts.
4. La red B cuenta con 60 hosts
5. La red Y tiene 46 hosts y se espera un crecimiento máximo de 18 hosts.
6. En cada red, se debe desperdiciar la menor cantidad de direcciones IP posibles. En este sentido, las redes utilizadas para conectar los routers deberán utilizar segmentos de red /30 de modo de desperdiciar la menor cantidad posible de direcciones IP.

Respuesta

Primeramente, ordeno la cantidad de host que requiere cada red en orden descendente:

- Red A 125 hosts y se espera un crecimiento máximo de 20 hosts
- Red X tiene 63 hosts
- Red B cuenta con 60 hosts
- Red Y tiene 46 hosts se espera un crecimiento máximo de 18 hosts

Bloque: 200.100.8.0 /22

`11001000.01100100.00001000.00000000`
Máscara: `11111111.11111111.11111100.00000000`

Primer red

Primer red a subnetear es la red A.
Para poder representar 125 hosts y cubrir la necesidad de crecimiento, con la cual nos daría 145, como máximo. Debemos seleccionar una cantidad tal que 2^n sea mayor o igual que 145. En este caso 2^8 - 2 >= 145.
La máscara pasa de ser de /22 a ser de: 32 - 8 = 24.
Máscara: `11111111.11111111.11111111.00000000`

Mínimo: 200.100.8.0
Máximo: 200.100.8.255
IP libres:

- `11001000.01100100.00001010.00000000`
- `11001000.01100100.00001011.00000000`

Segunda red

La siguiente dirección a dividir es la 200.100.9.0/24. Vamos por la red Y, que tiene tendría como máximo 64 hosts. Para poder representarlo necesitamos 7 bits. Siendo $2^7 - 2 >= 64$
La máscara pasa a ser 32 - 7 = 25.

Dirección: `11001000.01100100.00001001.00000000`
Máscara: `11111111.11111111.11111111.10000000`

Mínimo: 200.100.9.0
Máximo: 200.100.9.127

Tercer red

La siguiente dirección a dividir es la 200.100.9.128/25. En este caso para la red X. La cual también necesitara 7 bits. Siendo $2^7 - 2 >= 63$

Dirección: `11001000.01100100.00001001.10000000`
Máscara: `11111111.11111111.11111111.10000000`

Minimo: 200.100.9.128
Máximo: 200.100.9.255

Cuarta red

La siguiente dirección a dividir es la 200.100.10.0/25. En este caso para la red B. La cual requiere 6 bits. Siendo $2^6 - 2 >= 60$
La máscara pasa de 25 a: 32 - 6 = 26

Dirección: `11001000.01100100.00001010.00000000`
Máscara: `11111111.11111111.11111111.11000000`

Minimo: 200.100.10.0
Máximo: 200.100.10.63

Próxima subred libre: 200.100.10.64/26

Subneteo de 5 enlaces routers

- Enlace `n1` — `n2`
- Enlace `n1` — `n3`
- Enlace `n1` — `n4`
- Enlace `n2` — `n4`
- Enlace `n3` — `n4`

Tomo la subred libre que dejo la red B, que es la: 200.100.10.64/26.
Para enlaces entre routers con 2 hosts útiles se usa máscara /**30** ($2^2 - 2 = 2$ hosts útiles, máscara `255.255.255.252`).
os bloques `/30` avanzan en saltos de **4** en el 4.° octeto ($256 - 252 = 4$):

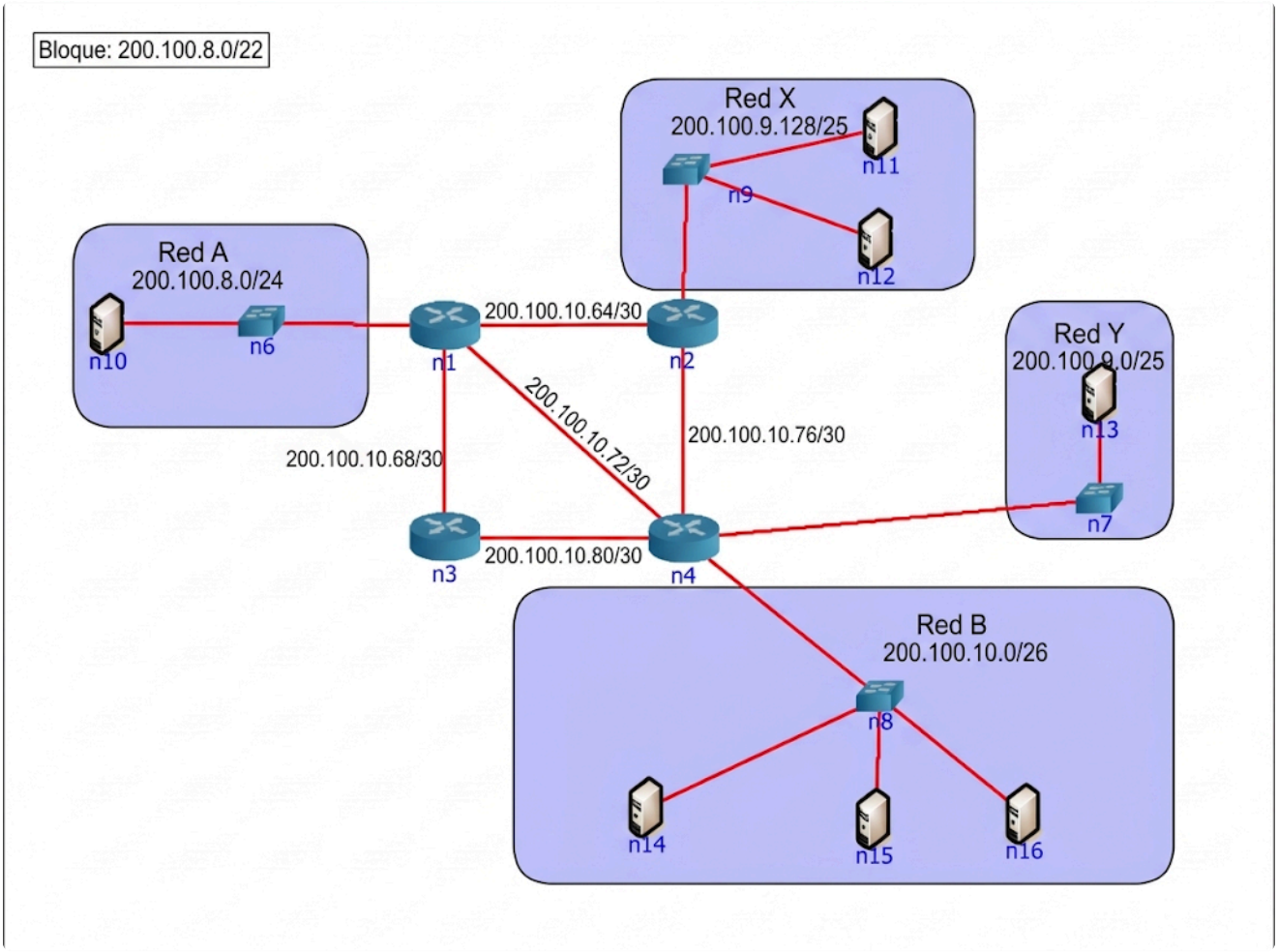
- **Enlace** `n1` — `n2` : `200.100.10.64/30`
  - Rango útil: `200.100.10.65` y `200.100.10.66`
  - Broadcast: `200.100.10.67`
- **Enlace** `n1` — `n3` : `200.100.10.68/30`
  - Rango útil: `200.100.10.69` y `200.100.10.70`
  - Broadcast: `200.100.10.71`

- **Enlace** `n1` — `n4`: `200.100.10.72/30`
  - Rango útil: `200.100.10.73` y `200.100.10.74`
  - Broadcast: `200.100.10.75`
- **Enlace** `n2` — `n4`: *`200.100.10.76/30`*
  - Rango útil: `200.100.10.77` y `200.100.10.78`
  - Broadcast: `200.100.10.79`
- **Enlace** `n3` — `n4`: `200.100.10.80/30`
  - Rango útil: `200.100.10.81` y `200.100.10.82`
  - Broadcast: `200.100.10.83`

Próxima subred libre para futuros enlaces: `200.100.10.84/30`

Asignación de IPS (Secundario)

- **Router** `n1`:
  - Interfaz hacia `n2`: `200.100.10.65/30`
  - Interfaz hacia `n3`: `200.100.10.69/30`
  - Interfaz hacia `n4`: `200.100.10.73/30`
- **Router** `n2`:
  - Interfaz hacia `n1`: `200.100.10.66/30`
  - Interfaz hacia `n4`: `200.100.10.77/30`
- **Router** `n3`:
  - Interfaz hacia `n1`: `200.100.10.70/30`
  - Interfaz hacia `n4`: `200.100.10.81/30`
- **Router** `n4`:
  - Interfaz hacia `n1`: `200.100.10.74/30`
  - Interfaz hacia `n2`: `200.100.10.78/30`
  - Interfaz hacia `n3`: `200.100.10.82/30`



Inciso 18

Asigne direcciones IP en los equipos de la topología según el plan anterior

Respuesta

Equipos de las Redes LAN (Hosts y Servidores)

- **Red A** (`200.100.8.0/24`):
  - **Servidor** `n10`: `200.100.8.2/24` (Gateway: `200.100.8.1`)
- **Red X** (`200.100.9.128/25`):
  - **Servidor** `n11`: `200.100.9.130/25` (Gateway: `200.100.9.129`)
  - **Servidor** `n12`: `200.100.9.131/25` (Gateway: `200.100.9.129`)
- **Red Y** (`200.100.9.0/25`):
  - **Servidor** `n13`: `200.100.9.2/25` (Gateway: `200.100.9.1`)
- **Red B** (`200.100.10.0/26`):
  - **Servidor** `n14`: `200.100.10.2/26` (Gateway: `200.100.10.1`)
  - **Servidor** `n15`: `200.100.10.3/26` (Gateway: `200.100.10.1`)
  - **Servidor** `n16`: `200.100.10.4/26` (Gateway: `200.100.10.1`)

2. Interfaces de los Routers

Router `n1`

- **Interfaz hacia switch** `n6` **(LAN Red A):** `200.100.8.1/24`
- **Interfaz hacia router** `n2` **:** `200.100.10.65/30`
- **Interfaz hacia router** `n3` **:** `200.100.10.69/30`
- **Interfaz hacia router** `n4` **:** `200.100.10.73/30`

Router `n2`

- **Interfaz hacia switch** `n9` **(LAN Red X):** `200.100.9.129/25`
- **Interfaz hacia router** `n1` **:** `200.100.10.66/30`
- **Interfaz hacia router** `n4` **:** `200.100.10.77/30`

Router `n3`

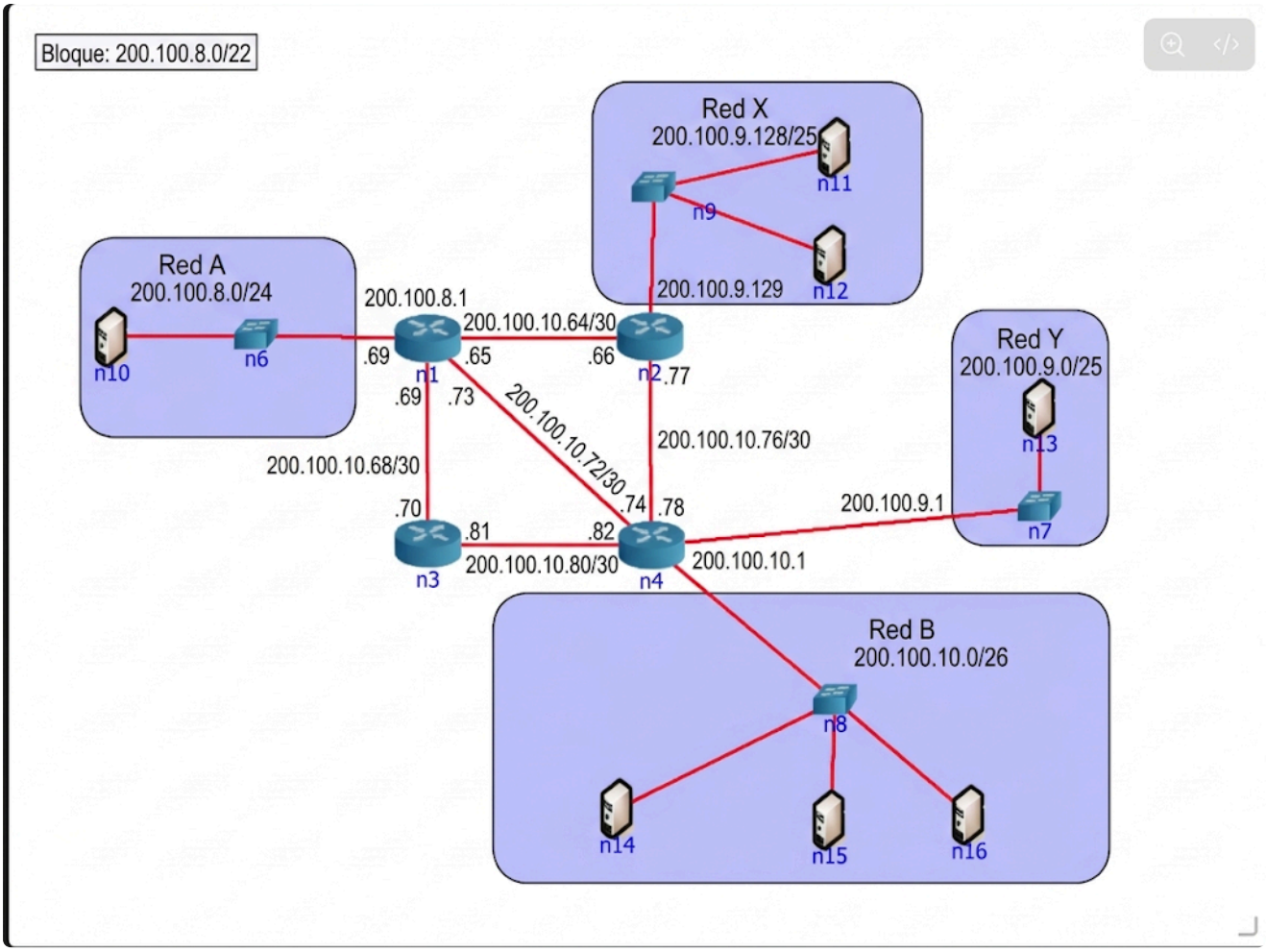
- **Interfaz hacia router** `n1` **:** `200.100.10.70/30`
- **Interfaz hacia router** `n4` **:** `200.100.10.81/30`

Router `n4`

- **Interfaz hacia switch** `n8` **(LAN Red B):** `200.100.10.1/26`
- **Interfaz hacia switch** `n7` **(LAN Red Y):** `200.100.9.1/25`
- **Interfaz hacia router** `n1` **:** `200.100.10.74/30`
- **Interfaz hacia router** `n2` **:** `200.100.10.78/30`
- **Interfaz hacia router** `n3` **:** `200.100.10.82/30`

3. Verificación de los Enlaces Punto a Punto entre Routers (`/30`)

- **Enlace** `n1` — `n2` (`200.100.10.64/30`): `n1` = `.65` | `n2` = `.66`
- **Enlace** `n1` — `n3` (`200.100.10.68/30`): `n1` = `.69` | `n3` = `.70`
- **Enlace** `n1` — `n4` (`200.100.10.72/30`): `n1` = `.73` | `n4` = `.74`
- **Enlace** `n2` — `n4` (`200.100.10.76/30`): `n2` = `.77` | `n4` = `.78`
- **Enlace** `n3` — `n4` (`200.100.10.80/30`): `n3` = `.81` | `n4` = `.82`



ICMP y Configuraciones IP

Inciso 19

Describa qué es y para qué sirve el protocolo ICMP.

1. Analice cómo funciona el comando ping.
    1. Indique el tipo y código ICMP que usa el ping.
    2. Indique el tipo y código ICMP que usa la respuesta de un ping.
2. Analice cómo funcionan comandos como traceroute/tracert de Linux/Windows y cómo manipulan el campo TTL de los paquetes IP.
3. Indique la cantidad de saltos realizados desde su computadora hasta el sitio WWW.NASA.GOV. Analice:
    1. Cómo hacer para que no muestre el nombre del dominio asociado a la IP de cada salto.
    2. La razón de la aparición de * en parte o toda la respuesta de un salto.
4. Verifique el recorrido hacia los servidores de nombre del dominio unlp.edu.ar. En base al recorrido realizado, ¿podría confirmar cuál de ellos toma un camino distinto?

Inciso 19.1

El comando **ping** es una herramienta de diagnóstico de red utilizada para verificar si un host remoto está activo y accesible, así como para medir el tiempo de ida y vuelta (**RTT**) de los paquetes. Funciona enviando mensajes de control del protocolo **ICMP** (*Internet Control Message Protocol*) encapsulados en datagramas IP hacia la dirección de destino. Al recibir la solicitud, el host destino responde y el cliente calcula la latencia y la pérdida de paquetes[1][2].

- **1.1. Tipo y código ICMP que usa el ping** (**solicitud**)**:** Utiliza una **Solicitud de Eco** (*Echo Request*), correspondiente al **Tipo 8, Código 0**.
- **1.2. Tipo y código ICMP que usa la respuesta de un ping :** El host de destino responde con una **Respuesta de Eco** (*Echo Reply*), correspondiente al **Tipo 0**, **Código 0**.

Inciso 19.2

El comando **traceroute** (en Linux) o **tracert** (en Windows) permite descubrir la secuencia de routers intermediarios (saltos) en la ruta hacia un destino y medir el retardo en cada uno de ellos.

Mecanismo y manipulación del campo TTL (**Time To Live**)**:**

1. El equipo origen envía un primer paquete hacia el destino configurando el campo **TTL** = **1** en la cabecera IP.
2. El primer router que recibe el paquete decrementa el TTL a 0. Siguiendo las reglas del protocolo IP, descarta el paquete y devuelve al origen un mensaje de advertencia **ICMP Time Exceeded** (**TTL caducado**) de **Tipo 11, Código 0**. El emisor registra la IP y nombre de este primer salto y calcula el RTT.
3. A continuación, el emisor envía un nuevo paquete con **TTL** = **2**. El primer router decrementa el TTL a 1 y lo reenvía al segundo router, donde el TTL llega a 0 y se genera otro mensaje **ICMP Time Exceeded** (Tipo 11, Código 0).
4. El proceso se repite incrementando el TTL de forma sucesiva (**TTL** = **3, 4, ...**) para identificar uno a uno todos los routers del camino.

Diferencia de implementación entre sistemas:

- **Linux** (**traceroute por defecto**)**:** Envía datagramas **UDP** dirigidos a un rango de puertos altos e improbables (ej. 33434+). Cuando un paquete llega

finalmente al host de destino, como no hay ninguna aplicación escuchando en ese puerto UDP, el destino responde con un mensaje **ICMP Destination Unreachable / Port Unreachable** (**Tipo 3, Código 3**). Al recibir este aviso, `traceroute` sabe que ha alcanzado el destino final y detiene la traza.

- **Windows (tracert):** Envía paquetes **ICMP Echo Request** (Tipo 8, Código 0) con TTL incremental[1][4]. Al llegar al destino final, este responde directamente con un **ICMP Echo Reply** (Tipo 0, Código 0).

Inciso 19.3

La cantidad exacta de saltos desde tu computadora depende de tu proveedor de Internet (ISP) y ubicación geográfica, ubicándose habitualmente entre 10 y 20 saltos.

- **3.1. Cómo evitar que se muestre el nombre de dominio asociado a cada IP:** Para deshabilitar la resolución inversa de DNS (*Reverse DNS lookup*) y agilizar la traza mostrando solo las direcciones IP:
  - En **Linux / macOS (traceroute):** Usar la opción **-n** (ejemplo: `traceroute -n www.nasa.gov`).
  - En **Windows (tracert):** Usar la opción **-d** (ejemplo: `tracert -d www.nasa.gov`).
- **3.2. Razón de la aparición de asteriscos (*) en la respuesta:** Un asterisco (`*`) representa un **tiempo de espera agotado (timeout)** al no recibir respuesta para ese intento de sondeo. Esto ocurre principalmente por:
  1. **Filtrado por Firewall / Descarte de ICMP:** El router o cortafuegos de ese salto está configurado para descartar paquetes de sondeo o para no emitir respuestas ICMP por políticas de seguridad.
  2. **Limitación de tasa (Rate Limiting):** El router limita la cantidad de mensajes ICMP que genera por segundo para evitar sobrecargar su CPU.
  3. **Pérdida de paquetes o congestión:** Pérdida del paquete de sondeo o del mensaje de respuesta ICMP en ese tramo de la red.

Inciso 19.4

Para consultar los servidores autoritativos del dominio se utiliza el comando `dig -t ns unlp.edu.ar` o `host -t ns unlp.edu.ar`. Entre sus servidores autoritativos se encuentran:

- **unlp.unlp.edu.ar** (`163.10.0.67`)
- **anubis.unlp.edu.ar** (`163.10.0.65`)
- **server.cespi.unlp.edu.ar** (`163.10.0.84`) *(además de servidores secundarios en redes externas de otras instituciones, como ARIU/RIU).*

Inciso 20

¿Para que se usa el bloque 127.0.0.0/8? ¿Qué PC responde a los siguientes comandos?

1. ping 127.0.0.1
2. ping 127.0.54.43

Respuesta

El bloque **127.0.0.0/8** está reservado para el **bucle local** (**loopback**) en IPv4.

- **Comunicación interna:** Permite que un host se envíe tráfico a sí mismo a través de una interfaz virtual de red interna (`lo`).
- **Pruebas y diagnóstico:** Sirve para verificar que la pila de protocolos TCP/IP del sistema operativo funciona correctamente sin requerir una placa de red física ni un cable conectado.
- **Servicios locales:** Permite que aplicaciones y servicios (servidores web, bases de datos, etc.) que corren en la misma máquina se comuniquen entre sí de manera segura y sin generar tráfico externo.

¿Qué PC responde a los comandos ping?

1. **ping 127.0.0.1**: Responde la **propia computadora local** (la misma PC desde la que ejecutás el comando).
2. **ping 127.0.54.43**: Responde la **misma computadora local**.

Razón: Cualquier dirección IP que comience con **127.** pertenece a la red de bucle local. Aunque habitualmente se usa `127.0.0.1` (asociada por defecto al nombre `localhost`), **todo el bloque /8 (desde 127.0.0.0 hasta 127.255.255.255) está redirigido internamente hacia el propio equipo**.

Inciso 21

Investigue para qué sirven los comandos ifconfig y route. ¿Qué comandos podría utilizar en su reemplazo? Inicie una topología con CORE, cree una

máquina y utilice en ella los comandos anteriores para practicar sus diferentes opciones, mínimamente:

- Configurar y quitar una dirección IP en una interfaz.
- Ver la tabla de ruteo de la máquina

Respuesta

Comandos más tradicionales:

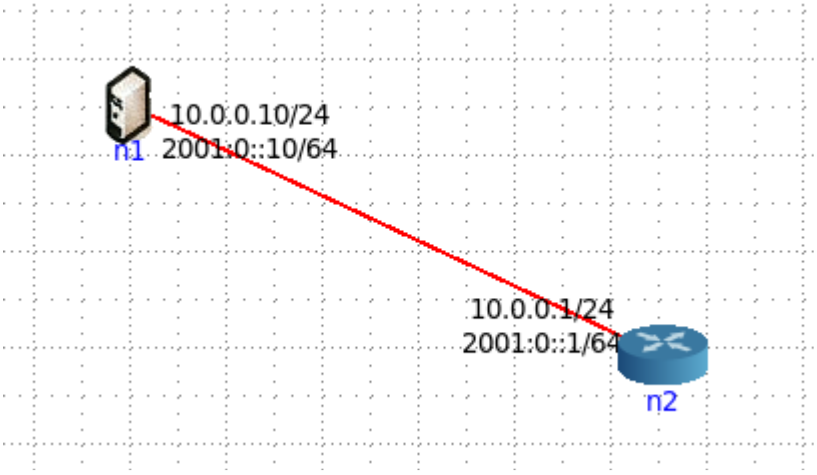
- `ifconfig` (***interface configuration***)**:** Herramienta clásica de Linux para visualizar el estado de las interfaces de red, asignar o modificar direcciones IP, máscaras de subred y activar o desactivar interfaces.
- `route`**:** Comando empleado para visualizar y manipular manualmente la tabla de rutas IP del núcleo (añadir o eliminar rutas estáticas hacia redes, hosts o la puerta de enlace predeterminada).
  Comandos más modernos:

Sus reemplazos directos pertenecen a la suite `iproute2`:

- En reemplazo de `ifconfig`: `ip addr` (o `ip link` para el estado del enlace).
- En reemplazo de `route`: `ip route`.

Función	**Comando tradicional** (`net-tools`)	**Comando moderno** (`iproute2`)
Ver direcciones e interfaces	`ifconfig`	`ip addr` (o `ip a`, `ip -br addr`)
Asignar IP a interfaz	`ifconfig eth0 <IP> netmask <MASCARA> up`	`ip addr add <IP>/<PREFIJO> dev eth0`
Quitar IP de interfaz	`ifconfig eth0 0.0.0.0` (o `down`)	`ip addr del <IP>/<PREFIJO> dev eth0`
Ver tabla de ruteo	`route -n`	`ip route` (o `ip route show`)
Agregar gateway predeterminado	`route add default gw <IP>`	`ip route add default via <IP>`

Ejercicio en CORE



Validar que las interfaces existan

Comando clásico:

```
root@n1:/tmp/pycore.45567/n1.conf# ifconfig -a
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.0.10  netmask 255.255.255.0  broadcast 0.0.0.0
        inet6 2001::10  prefixlen 64  scopeid 0x0<global>
        inet6 fe80::200:ff:feaa:0  prefixlen 64  scopeid 0x20<link>
        ether 00:00:00:aa:00:00  txqueuelen 1000  (Ethernet)
        RX packets 83  bytes 9218 (9.0 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 12  bytes 1032 (1.0 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

root@n1:/tmp/pycore.45567/n1.conf#
```

Comando más "moderno":

```
root@n1:/tmp/pycore.45567/n1.conf# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
14: eth0@if15: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default qlen 1000
    link/ether 00:00:00:aa:00:00 brd ff:ff:ff:ff:ff:ff link-netnsid 0
root@n1:/tmp/pycore.45567/n1.conf#
```

Ejecución de comandos

```
root@n1:/tmp/pycore.45567/n1.conf# ifconfig eth0 192.168.1.50 netmask 255.255.255.0 up
root@n1:/tmp/pycore.45567/n1.conf# route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     0.0.0.0         255.255.255.0   U     0      0        0 eth0
root@n1:/tmp/pycore.45567/n1.conf#
```

O con la suite moderna: `ip addr add 192.168.1.50/24 dev eth0` y `ip link set eth0 up`

Sacar la ip asignada:

